Guía 2 de 3 — APIs: pedir datos a internet

DPL1046 · Clase 4 · Para repasar en casa, a tu ritmo.

Hoy los datos no siempre están en un archivo: muchas veces hay que **pedirlos a un servicio** por internet (una API). Esta guía repasa qué es una API, cómo hacer un pedido con `requests`, qué hacer cuando falla, y cómo convertir la respuesta en una tabla.

> Antes de empezar, haz tu rutina de inicio (Guía 1 de la Clase 2) con el `.venv` activado. Necesitas `requests` (viene con `pip install -r clases/clase-04/material/requirements.txt`) y conexión a internet.

---

1. ¿Qué es una API? El garzón del restaurante

Una **API** es un **garzón**. Tú (tu programa) estás sentado en la mesa; en la cocina están los datos. No entras a la cocina: le haces un **pedido** al garzón y él te trae la **respuesta**. Pedido claro, respuesta ordenada, y no necesitas saber cómo funciona la cocina.

Restaurante	API
Comensal	Tu script de Python
Garzón	La API
Cocina	El servidor con los datos
Pedido	La petición (`requests.get`)
Plato	La respuesta (JSON)

Un pedido con `requests`

```
import requests

URL = "https://mindicador.cl/api"

try:
    respuesta = requests.get(URL, timeout=10)
    respuesta.raise_for_status()
    print("Código de respuesta:", respuesta.status_code)

    datos = respuesta.json()
    print("Dólar hoy:", datos["dolar"]["valor"])
    print("UF hoy:", datos["uf"]["valor"])
except requests.exceptions.RequestException:
    print("No pude conectar. Revisa tu internet.")
```

Línea por línea:

- `requests.get(URL, timeout=10)` → "dame los datos de esta dirección; espera máximo 10 segundos".
- `raise_for_status()` → si el servidor respondió con error, lo avisa.
- `status_code` → el **código**: `200` = todo bien; `404` = esa dirección no existe; `500` = falló el servidor (no es culpa tuya).
- `respuesta.json()` → lo que llega es **texto**; `.json()` lo convierte en un **diccionario**.
- `datos["dolar"]["valor"]` → acceso anidado, igual que en la Guía 1: edificio `dolar`, buzón `valor`.

`try / except`: el paraguas

`try` significa "intenta esto". Si todo sale bien, el `except` se ignora. Si falla (no hay internet, la API está caída, tarda demasiado), Python **salta al `except`** y muestra un mensaje claro en vez de llenar la pantalla de letras rojas. Internet puede fallar, y es normal.

De varios indicadores a una tabla

```
filas = []
for nombre in ["dolar", "euro", "uf", "utm"]:
    filas.append({"indicador": nombre, "valor": datos[nombre]["valor"]})

tabla = pd.DataFrame(filas)
print(tabla)
```

- `filas = []` → una bandeja vacía.
- `for nombre in [...]` → "para cada indicador" (`nombre` vale `dolar`, luego `euro`...).
- `append({...})` → agrega un diccionario por indicador.
- `pd.DataFrame(filas)` → una **lista de diccionarios** se convierte en **tabla**: cada diccionario es una fila.

---

2. Cómo conecta con tu proyecto

Una API pública es una **segunda fuente de datos**: la clave para cumplir "dos o más fuentes o tablas" de la rúbrica. En la presentación va en la slide 3 (**Los datos**) y en la slide 4 (**Arquitectura**). Piensa: ¿qué dato de tu tema podría venir de una API? (el dólar, la UF, el clima, una API pública de tu rubro).

> Las **claves de una API** y los **datos personales** los vemos en la **Clase 5**.

Errores frecuentes

Qué ves	Por qué pasa	Qué hacer
`ConnectionError` / `timeout`	Sin internet o API lenta	Revisa tu conexión; el `except` debe avisar sin romper el programa
`KeyError` al leer el JSON	La clave no existe en la respuesta	Imprime `datos` y revisa los nombres exactos
`404`	La dirección está mal escrita	Revisa la URL letra por letra
`500`	Falló el servidor	No es tu culpa: espera e intenta más tarde
`ModuleNotFoundError: requests`	Falta instalar	Activa el `.venv` y ejecuta `pip install -r clases/clase-04/material/requirements.txt`

Practica tú

1. ¿Cómo pides el valor del **euro** con el JSON `datos` de arriba?
2. ¿Qué significa un código de respuesta `404`? ¿Y `200`?
3. ¿Cuántas filas tendrá la tabla `tabla` del ejemplo? ¿Por qué?
4. ¿Qué hace `timeout=10`?

▼ **Ver respuestas**

1. `datos["euro"]["valor"]`
2. `404`: la dirección no existe. `200`: todo bien.
3. 4 filas: una por cada indicador de la lista (`dolar`, `euro`, `uf`, `utm`).
4. Espera como máximo 10 segundos la respuesta; si no llega, corta y salta al `except`.

Para llevarte

- Una **API** es un garzón: pedido → respuesta (JSON → diccionario con `.json()`).
- Siempre `timeout` y `try / except`: internet falla.
- Una **lista de diccionarios** se convierte en tabla con `pd.DataFrame(...)`.
- Una API pública es una segunda fuente para tu proyecto.